
RetailPulse

Sales Performance Dashboard

Capstone Project Report
SQL and Database Management Systems
EduSkills Academy

Submitted by: Prathamesh Chaunmwal
Student ID: SOE24201030044
University: Pimpri Chinchwad University
Programme: SQL and Database Management Systems
Partner: EduSkills Academy
Date: June 1, 2026

PostgreSQL 15 • Node.js / Express.js • Power BI Desktop

Abstract

RetailPulse is a comprehensive, end-to-end Sales Performance Dashboard built as the capstone project for the EduSkills SQL and Database Management Systems programme. The project simulates a real-world business intelligence solution for a mid-sized retail company operating across multiple geographical regions.

The system encompasses the complete data analytics pipeline: a normalised relational database hosted on PostgreSQL, a RESTful backend API built with Node.js and Express.js, and an interactive multi-page dashboard built in Power BI Desktop. The database stores transactional sales data across six interrelated tables covering regions, product categories, products, customers, orders, and order items.

The backend API exposes eight analytical endpoints providing aggregated sales metrics including total revenue, regional performance, category breakdowns, top products, and monthly sales trends. The Power BI dashboard presents these insights across three interactive pages — Executive Summary, Regional Performance, and Product Performance — enabling stakeholders at all levels to make data-driven decisions.

Keywords: PostgreSQL, Node.js, Express.js, Power BI, REST API, Business Intelligence, Sales Analytics, Star Schema, Data Visualisation.

Contents

Abstract	1
1 Project Overview	6
1.1 Introduction	6
1.2 Project Goal	6
1.3 Business Scenario	6
1.4 Key Performance Indicators (KPIs)	7
1.5 Technology Stack	7
1.6 Project Deliverables	7
2 Database Design & Implementation	8
2.1 Database Overview	8
2.2 Entity-Relationship Diagram	8
2.3 Schema Design	9
2.3.1 Table Relationships	9
2.3.2 DDL Scripts	9
2.3.3 Screenshot — Tables Created	11
2.4 Data Population	11
2.5 Data Dictionary	13
2.5.1 Regions	13
2.5.2 Categories	13
2.5.3 Products	13
2.5.4 Customers	13
2.5.5 Orders	14
2.5.6 Order_Items	14
3 Backend API Development	16
3.1 API Overview	16
3.2 Dependencies	16
3.3 Environment Configuration	16
3.4 Database Connection	17
3.5 Server Setup	17
3.6 API Endpoints	18
3.7 Endpoint Implementations	18
3.8 API Screenshots	20
4 Power BI Dashboard	24
4.1 Dashboard Overview	24

4.2	Data Model in Power BI	24
4.3	Design Theme	25
4.4	Dashboard Pages	25
4.4.1	Page 1 — Executive Summary	25
4.4.2	Page 2 — Regional Performance	27
4.4.3	Page 3 — Product Performance	28
4.5	Key Insights from the Dashboard	29
5	Conclusion	30
5.1	Summary	30
5.2	Challenges and Learnings	30
5.3	Future Enhancements	31
5.4	Final Remarks	31

List of Figures

2.1	RetailPulse Entity-Relationship Diagram showing all six tables and their relationships.	8
2.2	\dt output confirming all six tables created in <code>retailpulse_db</code>	11
2.3	SELECT COUNT(*) results confirming all records were inserted correctly across all six tables.	12
3.1	RetailPulse API server running on <code>http://localhost:3000</code>	20
3.2	Root endpoint GET / confirming the API is active.	21
3.3	GET /api/sales/revenue-by-region returning aggregated revenue data per region.	22
3.4	Additional API endpoint output showing top products.	23
4.1	Power BI Model View showing all six tables and their auto-detected relationships.	25
4.2	Initial layout of the Executive Summary page showing KPI cards and Monthly Sales Trend before formatting.	26
4.3	Final Executive Summary page — KPI cards showing total revenue of 9.91K, 80 orders, 20 customers, and average order value of 123.88. Monthly Sales Trend shows peaks in February, April, November, and December. . .	27
4.4	Initial layout of the Regional Performance page before formatting.	27
4.5	Final Regional Performance page — North America West leads with 3.3K revenue, followed by Asia Pacific at 2.8K. USA dominates country-level revenue at 4.4K.	28
4.6	Initial layout of the Product Performance page before formatting.	28
4.7	Final Product Performance page — Laptop Pro 15 is the top revenue product at 2.6K. Electronics accounts for 55.5% of total category revenue. .	29

List of Tables

1.1	RetailPulse Key Performance Indicators	7
1.2	RetailPulse technology stack	7
2.1	Foreign key relationships in retailpulse_db	9
2.2	Record counts per table after data population	11
2.3	Data dictionary — Regions table	13
2.4	Data dictionary — Categories table	13
2.5	Data dictionary — Products table	13
2.6	Data dictionary — Customers table	14
2.7	Data dictionary — Orders table	14
2.8	Data dictionary — Order_Items table (Fact Table)	15
3.1	RetailPulse API — project structure	16
3.2	npm packages used in RetailPulse API	16
3.3	RetailPulse API endpoint reference	18
4.1	Dashboard pages and their contents	24
4.2	RetailPulse Power BI colour palette	25
5.1	Key challenges encountered and how they were resolved	30

Chapter 1

Project Overview

1.1 Introduction

RetailPulse is a data-driven Sales Performance Dashboard developed as the capstone project for the EduSkills SQL and Database Management Systems programme. The project consolidates skills acquired throughout the course, spanning relational database design, data population, backend API development, and interactive data visualisation.

The project simulates a real-world scenario in which a mid-sized retail company operating across multiple geographical regions requires a centralised, interactive dashboard to monitor and analyse sales performance. The company's current infrastructure relies on fragmented spreadsheets and manual reporting, making timely and accurate decision-making difficult.

1.2 Project Goal

The overarching goal of RetailPulse is to build a comprehensive, interactive Sales Performance Dashboard that:

- Provides a single source of truth for sales data across all regions and product categories.
- Enables stakeholders to track key performance indicators (KPIs) such as total revenue, order count, customer count, and average order value.
- Visualises sales trends over time to identify seasonality and growth patterns.
- Supports interactive filtering by date, region, and product category for ad-hoc analysis.
- Demonstrates end-to-end data management skills from database design to visualisation.

1.3 Business Scenario

The project is set within the context of a mid-sized retail company selling products across eight global regions spanning North America, Europe, Asia Pacific, South Asia, the Middle East, and Latin America. The company sells products across eight categories including Electronics, Clothing, Home Goods, Books, Sports, Beauty, Food & Beverage, and Toys.

Key business questions the dashboard aims to answer:

- What is the total revenue generated across all regions and time periods?

- Which regions and countries are the top revenue contributors?
- Which products and categories generate the most sales?
- How are sales trending month-over-month?
- What is the average order value across all transactions?

1.4 Key Performance Indicators (KPIs)

Table 1.1. RetailPulse Key Performance Indicators

KPI	Definition	Target
Total Revenue	Sum of all order item values (<code>quantity * price_at_purchase</code>)	Maximise YoY
Total Orders	Count of all orders placed	Track monthly
Total Customers	Count of unique registered customers	Grow base
Avg Order Value	Average revenue per order	Increase by 8%
Revenue by Region	Revenue breakdown per geographical region	Identify top regions
Revenue by Category	Revenue per product category	Balance portfolio

1.5 Technology Stack

Table 1.2. RetailPulse technology stack

Layer	Technology	Purpose
Database	PostgreSQL 15	Relational data storage and querying
API	Node.js + Express.js	RESTful backend data serving
ORM/Driver	pg-promise	PostgreSQL client for Node.js
Config	dotenv	Environment variable management
CORS	cors	Cross-origin request handling
Dashboard	Power BI Desktop	Interactive data visualisation

1.6 Project Deliverables

1. **PostgreSQL Database** — normalised schema with 6 tables, constraints, foreign keys, and realistic sample data.
2. **Backend REST API** — Node.js/Express.js server with 8 analytical endpoints.
3. **Power BI Dashboard** — 3-page interactive dashboard with KPI cards, charts, slicers, and page navigation.
4. **Project Documentation** — this report, including ERD, data dictionary, API reference, and dashboard screenshots.

Chapter 2

Database Design & Implementation

2.1 Database Overview

The RetailPulse database (`retailpulse_db`) is hosted on a local PostgreSQL 15 instance. It follows a normalised relational schema designed to support efficient analytical queries across sales transactions, products, customers, and regions.

The schema consists of six tables organised in a star-like structure with `Order_Items` as the central fact table surrounded by dimension tables.

2.2 Entity-Relationship Diagram

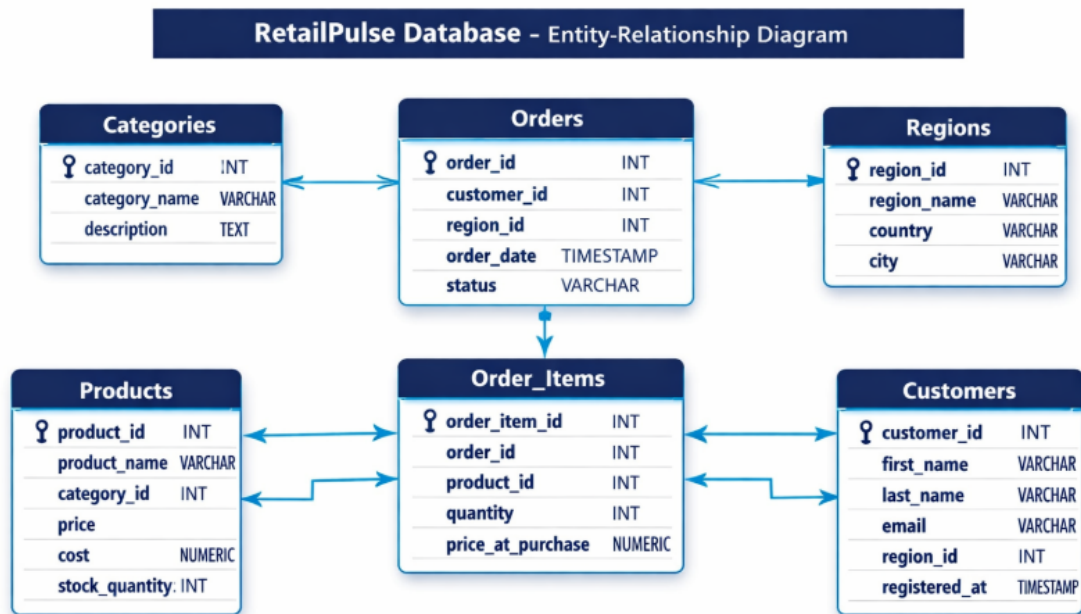


Figure 2.1. RetailPulse Entity-Relationship Diagram showing all six tables and their relationships.

2.3 Schema Design

2.3.1 Table Relationships

Table 2.1. Foreign key relationships in retailpulse_db

From Table	To Table	Cardinality	On Delete
Customers	Regions	Many to One (*:1)	Restrict
Orders	Customers	Many to One (*:1)	Restrict
Orders	Regions	Many to One (*:1)	Restrict
Products	Categories	Many to One (*:1)	Restrict
Order_Items	Orders	Many to One (*:1)	Cascade
Order_Items	Products	Many to One (*:1)	Restrict

2.3.2 DDL Scripts

```
1 CREATE DATABASE retailpulse_db;
2 \c retailpulse_db
```

Listing 2.1. Create database and connect

```
1 CREATE TABLE Regions (
2   region_id SERIAL PRIMARY KEY,
3   region_name VARCHAR(100) NOT NULL,
4   country VARCHAR(100) NOT NULL,
5   city VARCHAR(100) NOT NULL
6 );
```

Listing 2.2. Table 1 — Regions

```
1 CREATE TABLE Categories (
2   category_id SERIAL PRIMARY KEY,
3   category_name VARCHAR(100) NOT NULL,
4   description TEXT
5 );
```

Listing 2.3. Table 2 — Categories

```
1 CREATE TABLE Products (
2   product_id SERIAL PRIMARY KEY,
3   product_name VARCHAR(255) NOT NULL,
4   category_id INT NOT NULL REFERENCES Categories(
5     category_id),
6   price NUMERIC(10,2) NOT NULL CHECK (price >=
7     0),
8   cost NUMERIC(10,2) NOT NULL CHECK (cost >=
9     0),
10  stock_quantity INT NOT NULL DEFAULT 0
11 );
```

Listing 2.4. Table 3 — Products

```
1 CREATE TABLE Customers (  
2   customer_id SERIAL PRIMARY KEY,  
3   first_name VARCHAR(100) NOT NULL,  
4   last_name VARCHAR(100) NOT NULL,  
5   email VARCHAR(255) UNIQUE NOT NULL,  
6   region_id INT NOT NULL REFERENCES Regions(  
       region_id),  
7   registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
8 );
```

Listing 2.5. Table 4 — Customers

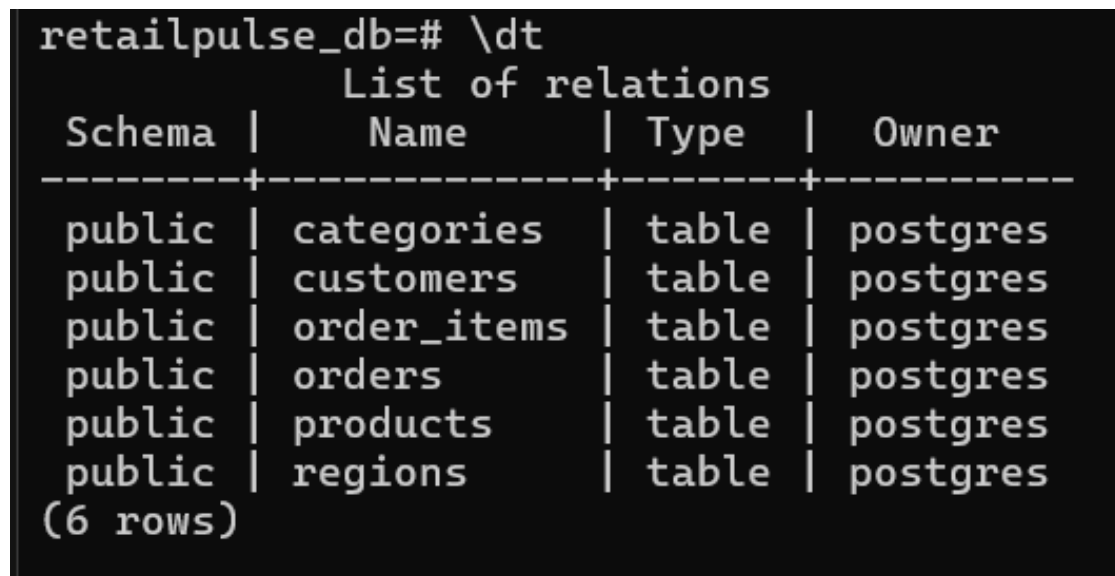
```
1 CREATE TABLE Orders (  
2   order_id SERIAL PRIMARY KEY,  
3   customer_id INT NOT NULL REFERENCES Customers(  
       customer_id)  
4   ON DELETE RESTRICT,  
5   region_id INT NOT NULL REFERENCES Regions(region_id)  
6   ,  
7   order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
8   status VARCHAR(50) DEFAULT 'Completed'  
9 );
```

Listing 2.6. Table 5 — Orders

```
1 CREATE TABLE Order_Items (  
2   order_item_id SERIAL PRIMARY KEY,  
3   order_id INT NOT NULL  
4   REFERENCES Orders(order_id) ON DELETE CASCADE,  
5   product_id INT NOT NULL  
6   REFERENCES Products(product_id) ON DELETE RESTRICT,  
7   quantity INT NOT NULL CHECK (quantity > 0),  
8   price_at_purchase NUMERIC(10,2) NOT NULL CHECK (  
       price_at_purchase >= 0)  
9 );
```

Listing 2.7. Table 6 — Order_Items (Fact Table)

2.3.3 Screenshot — Tables Created



```
retailpulse_db=# \dt
               List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | categories      | table | postgres
public | customers        | table | postgres
public | order_items      | table | postgres
public | orders           | table | postgres
public | products         | table | postgres
public | regions          | table | postgres
(6 rows)
```

Figure 2.2. \dt output confirming all six tables created in retailpulse_db.

2.4 Data Population

The database was populated with realistic sample data covering:

Table 2.2. Record counts per table after data population

Table	Description	Records
Regions	Global sales regions across 7 countries	8
Categories	Product categories	8
Products	Individual products across all categories	40
Customers	Registered customers across all regions	20
Orders	Sales orders from Jan 2023 to Jul 2024	40
Order_Items	Individual line items per order	80

```
retailpulse_db=# SELECT COUNT(*) FROM Regions;
count
-----
      8
(1 row)

retailpulse_db=# SELECT COUNT(*) FROM Categories;
count
-----
      8
(1 row)

retailpulse_db=# SELECT COUNT(*) FROM Products;
count
-----
     40
(1 row)

retailpulse_db=# SELECT COUNT(*) FROM Customers;
count
-----
     20
(1 row)

retailpulse_db=# SELECT COUNT(*) FROM Orders;
count
-----
     40
(1 row)

retailpulse_db=# SELECT COUNT(*) FROM Order_Items;
count
-----
     80
(1 row)
```

Figure 2.3. SELECT COUNT(*) results confirming all records were inserted correctly across all six tables.

2.5 Data Dictionary

2.5.1 Regions

Column	Type	Constraints	Description
region_id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier
region_name	VARCHAR(100)	NOT NULL	Name of the sales region (e.g. North America East)
country	VARCHAR(100)	NOT NULL	Country the region belongs to
city	VARCHAR(100)	NOT NULL	Primary city of the region

Table 2.3. Data dictionary — Regions table

2.5.2 Categories

Column	Type	Constraints	Description
category_id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier
category_name	VARCHAR(100)	NOT NULL	Name of the product category (e.g. Electronics)
description	TEXT	NULLABLE	Optional description of the category

Table 2.4. Data dictionary — Categories table

2.5.3 Products

Column	Type	Constraints	Description
product_id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier
product_name	VARCHAR(255)	NOT NULL	Name of the product
category_id	INT	FK → Categories, NOT NULL	Links product to its category
price	NUMERIC(10,2)	NOT NULL, CHECK ≥ 0	Selling price of the product
cost	NUMERIC(10,2)	NOT NULL, CHECK ≥ 0	Cost price of the product
stock_quantity	INT	NOT NULL, DEFAULT 0	Current stock level

Table 2.5. Data dictionary — Products table

2.5.4 Customers

Column	Type	Constraints	Description
customer_id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier
first_name	VARCHAR(100)	NOT NULL	Customer's first name
last_name	VARCHAR(100)	NOT NULL	Customer's last name
email	VARCHAR(255)	UNIQUE, NOT NULL	Customer's unique email address
region_id	INT	FK → Regions, NOT NULL	Region where the customer is based
registered_at	TIMESTAMP	DEFAULT NOW()	Date and time of registration

Table 2.6. Data dictionary — Customers table

2.5.5 Orders

Column	Type	Constraints	Description
order_id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier
customer_id	INT	FK → Customers, NOT NULL	Customer who placed the order
region_id	INT	FK → Regions, NOT NULL	Region where order was placed
order_date	TIMESTAMP	DEFAULT NOW()	Date and time the order was placed
status	VARCHAR(50)	DEFAULT 'Completed'	Current status of the order

Table 2.7. Data dictionary — Orders table

2.5.6 Order_Items

Column	Type	Constraints	Description
order_item_id	SERIAL	PRIMARY KEY	Auto-incrementing unique identifier
order_id	INT	FK → Orders, ON DELETE CASCADE, NOT NULL	Parent order reference
product_id	INT	FK → Products, ON DELETE RESTRICT, NOT NULL	Product included in the order
quantity	INT	NOT NULL, CHECK > 0	Number of units ordered

Column	Type	Constraints	Description
price_at_purchase	NUMERIC(10,2)	NOT NULL, CHECK ≥ 0	Price per unit at time of purchase

Table 2.8. Data dictionary — Order_Items table (Fact Table)

Chapter 3

Backend API Development

3.1 API Overview

The RetailPulse backend API is a RESTful service built using **Node.js** and **Express.js**, connected to the PostgreSQL database via the **pg-promise** library. It serves aggregated sales data to the Power BI dashboard and any other clients.

Table 3.1. RetailPulse API — project structure

File	Purpose
<code>server.js</code>	Main entry point; registers middleware and mounts routes
<code>db.js</code>	pg-promise database connection configuration
<code>routes/salesRoutes.js</code>	All 8 sales analytics endpoints
<code>.env</code>	Environment variables (credentials, port)
<code>package.json</code>	Project metadata and dependencies

3.2 Dependencies

Table 3.2. npm packages used in RetailPulse API

Package	Version	Purpose
<code>express</code>	4.x	Web framework for routing and middleware
<code>pg-promise</code>	12.x	PostgreSQL client with promise support
<code>dotenv</code>	16.x	Load environment variables from <code>.env</code> file
<code>cors</code>	2.x	Enable Cross-Origin Resource Sharing

3.3 Environment Configuration

```
DB_USER=postgres
DB_PASSWORD="your_password"
DB_HOST=localhost
DB_PORT=5432
DB_NAME=retailpulse_db
```

```
PORT=3000
```

Listing 3.1. .env — environment variables**Note**

Passwords containing special characters (e.g. @, #, !) must be wrapped in double quotes in the .env file to ensure they are parsed correctly by the dotenv library.

3.4 Database Connection

```
1  require('dotenv').config();
2  const pgp = require('pg-promise')();
3
4  const db = pgp({
5      user:      process.env.DB_USER,
6      password: process.env.DB_PASSWORD,
7      host:      process.env.DB_HOST,
8      port:      process.env.DB_PORT,
9      database: process.env.DB_NAME
10 });
11
12 module.exports = db;
```

Listing 3.2. db.js — pg-promise connection setup

3.5 Server Setup

```
1  require('dotenv').config();
2  const express      = require('express');
3  const cors         = require('cors');
4  const salesRoutes  = require('./routes/salesRoutes');
5
6  const app = express();
7  const PORT = process.env.PORT || 3000;
8
9  app.use(cors());
10 app.use(express.json());
11 app.use('/api/sales', salesRoutes);
12
13 app.get('/', (req, res) => {
14     res.json({ message: 'RetailPulse API is running!' });
15 });
16
17 app.listen(PORT, () => {
18     console.log('RetailPulse API running on http://localhost'
19                 :`${PORT}`);
19 });
```

Listing 3.3. server.js — Express app entry point

3.6 API Endpoints

Table 3.3. RetailPulse API endpoint reference

Method	Endpoint	Description
GET	/	Health check — API status
GET	/api/sales/total-revenue	Total revenue across all orders
GET	/api/sales/total-orders	Total number of orders
GET	/api/sales/total-customers	Total number of customers
GET	/api/sales/avg-order-value	Average revenue per order
GET	/api/sales/revenue-by-region	Revenue grouped by region
GET	/api/sales/revenue-by-category	Revenue grouped by category
GET	/api/sales/top-products	Top 5 products by revenue
GET	/api/sales/monthly-sales	Monthly revenue trend

3.7 Endpoint Implementations

```

1  router.get('/total-revenue', async (req, res) => {
2    try {
3      const result = await db.one(`
4        SELECT ROUND(SUM(oi.quantity * oi.price_at_purchase)::
5          numeric, 2)
6        AS total_revenue FROM Order_Items oi
7        `);
8      res.json(result);
9    } catch (err) { res.status(500).json({ error: err.
      message }); }
10  });

```

Listing 3.4. GET /api/sales/total-revenue

```

1  router.get('/revenue-by-region', async (req, res) => {
2    try {
3      const result = await db.any(`
4        SELECT
5          r.region_name,
6          r.country,
7          ROUND(SUM(oi.quantity * oi.price_at_purchase)::numeric
8            , 2)
9          AS total_revenue
10         FROM Orders o
11        JOIN Regions r      ON o.region_id  = r.region_id
12        JOIN Order_Items oi ON o.order_id   = oi.order_id
13        GROUP BY r.region_name, r.country
14        ORDER BY total_revenue DESC
15        `);
16      res.json(result);
17    } catch (err) { res.status(500).json({ error: err.
      message }); }

```

```
17    });
```

Listing 3.5. GET /api/sales/revenue-by-region

```
1    router.get('/revenue-by-category', async (req, res) => {
2      try {
3        const result = await db.any(`
4          SELECT
5            c.category_name,
6            ROUND(SUM(oi.quantity * oi.price_at_purchase)::numeric
7              , 2)
8            AS total_revenue
9          FROM Order_Items oi
10         JOIN Products p    ON oi.product_id = p.product_id
11         JOIN Categories c ON p.category_id = c.category_id
12         GROUP BY c.category_name
13         ORDER BY total_revenue DESC
14       `);
15       res.json(result);
16     } catch (err) { res.status(500).json({ error: err.
17       message }); }
```

Listing 3.6. GET /api/sales/revenue-by-category

```
1    router.get('/top-products', async (req, res) => {
2      try {
3        const result = await db.any(`
4          SELECT
5            p.product_name,
6            SUM(oi.quantity) AS total_quantity,
7            ROUND(SUM(oi.quantity * oi.price_at_purchase)::numeric
8              , 2)
9            AS total_revenue
10         FROM Order_Items oi
11         JOIN Products p ON oi.product_id = p.product_id
12         GROUP BY p.product_name
13         ORDER BY total_revenue DESC
14         LIMIT 5
15       `);
16       res.json(result);
17     } catch (err) { res.status(500).json({ error: err.
18       message }); }
```

Listing 3.7. GET /api/sales/top-products

```
1    router.get('/monthly-sales', async (req, res) => {
2      try {
3        const result = await db.any(`
4          SELECT
5            TO_CHAR(o.order_date, 'YYYY-MM') AS month,
```

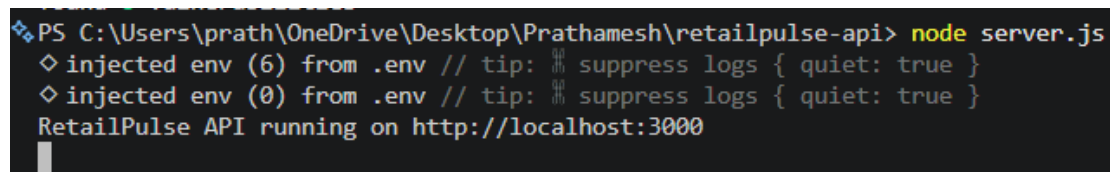
```
6      ROUND(SUM(oi.quantity * oi.price_at_purchase)::numeric
7            , 2)
8      AS monthly_revenue
9      FROM Orders o
10     JOIN Order_Items oi ON o.order_id = oi.order_id
11     GROUP BY TO_CHAR(o.order_date, 'YYYY-MM')
12     ORDER BY month ASC
13   ');
14   res.json(result);
15 } catch (err) { res.status(500).json({ error: err.
    message }); }
});
```

Listing 3.8. GET /api/sales/monthly-sales

```
1   router.get('/avg-order-value', async (req, res) => {
2     try {
3       const result = await db.one('
4         SELECT ROUND(AVG(order_total)::numeric, 2) AS
5           avg_order_value
6         FROM (
7           SELECT order_id,
8             SUM(quantity * price_at_purchase) AS order_total
9           FROM Order_Items
10          GROUP BY order_id
11         ) AS order_totals
12       ');
13       res.json(result);
14     } catch (err) { res.status(500).json({ error: err.
        message }); }
15   });
```

Listing 3.9. GET /api/sales/avg-order-value

3.8 API Screenshots



```
PS C:\Users\prath\OneDrive\Desktop\Prathamesh\retailpulse-api> node server.js
◇ injected env (6) from .env // tip: 🛑 suppress logs { quiet: true }
◇ injected env (0) from .env // tip: 🛑 suppress logs { quiet: true }
RetailPulse API running on http://localhost:3000
```

Figure 3.1. RetailPulse API server running on `http://localhost:3000`.

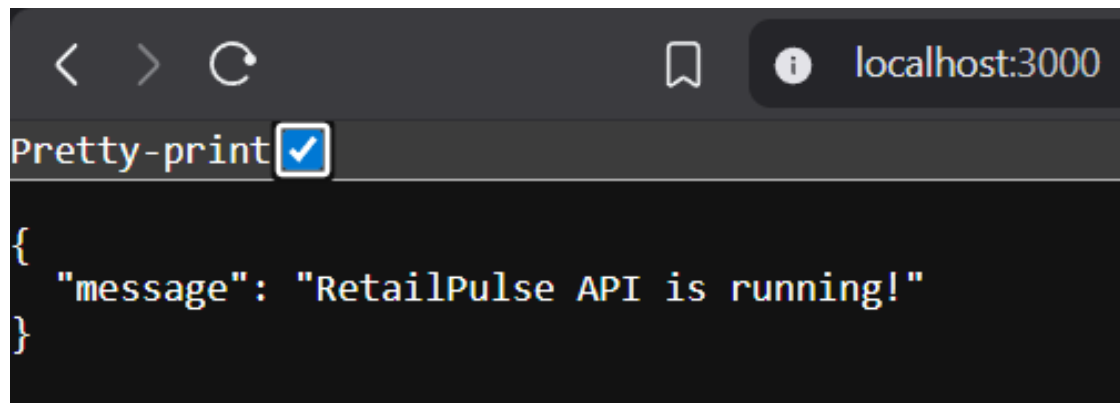
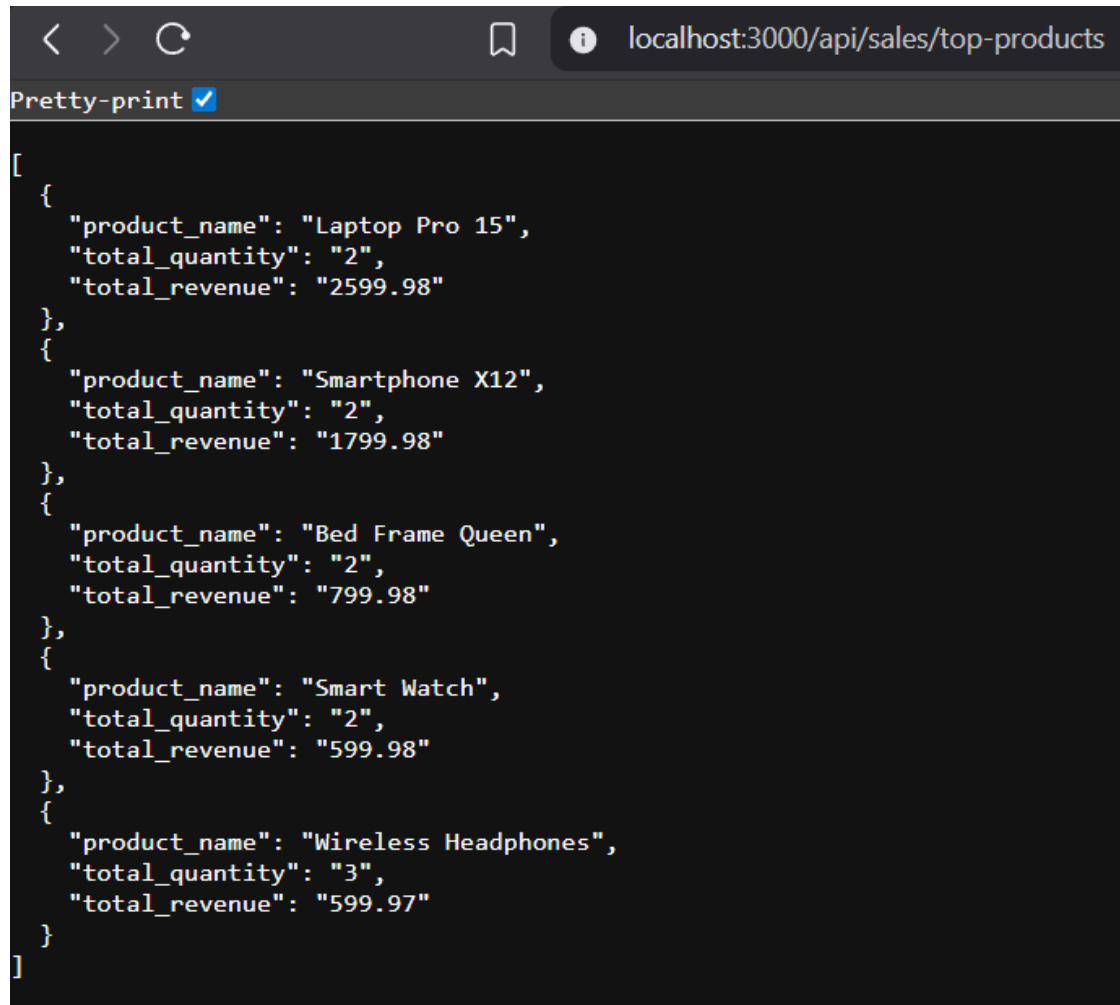


Figure 3.2. Root endpoint GET / confirming the API is active.



```
[
  {
    "region_name": "North America West",
    "country": "USA",
    "total_revenue": "3484.83"
  },
  {
    "region_name": "Asia Pacific",
    "country": "Japan",
    "total_revenue": "3084.84"
  },
  {
    "region_name": "North America East",
    "country": "USA",
    "total_revenue": "1479.83"
  },
  {
    "region_name": "Europe Central",
    "country": "Germany",
    "total_revenue": "1194.84"
  },
  {
    "region_name": "Europe West",
    "country": "UK",
    "total_revenue": "919.82"
  },
  {
    "region_name": "Middle East",
    "country": "UAE",
    "total_revenue": "724.87"
  },
  {
    "region_name": "South Asia",
    "country": "India",
    "total_revenue": "529.84"
  },
  {
    "region_name": "Latin America",
    "country": "Brazil",
    "total_revenue": "395.81"
  }
]
```

Figure 3.3. GET /api/sales/revenue-by-region returning aggregated revenue data per region.



```
[
  {
    "product_name": "Laptop Pro 15",
    "total_quantity": "2",
    "total_revenue": "2599.98"
  },
  {
    "product_name": "Smartphone X12",
    "total_quantity": "2",
    "total_revenue": "1799.98"
  },
  {
    "product_name": "Bed Frame Queen",
    "total_quantity": "2",
    "total_revenue": "799.98"
  },
  {
    "product_name": "Smart Watch",
    "total_quantity": "2",
    "total_revenue": "599.98"
  },
  {
    "product_name": "Wireless Headphones",
    "total_quantity": "3",
    "total_revenue": "599.97"
  }
]
```

Figure 3.4. Additional API endpoint output showing top products.

Chapter 4

Power BI Dashboard

4.1 Dashboard Overview

The RetailPulse Power BI Dashboard is a three-page interactive report built in Power BI Desktop, connected directly to the `retailpulse_db` PostgreSQL database. The dashboard follows a professional dark navy and cyan colour scheme consistent with the RetailPulse brand identity.

Table 4.1. Dashboard pages and their contents

Page	Name	Visuals
1	Executive Summary	4 KPI cards, Monthly Sales Trend line chart, Date slicer, Page navigator
2	Regional Performance	Revenue by Region bar chart, Revenue by Country column chart, Region slicer
3	Product Performance	Top 10 Products bar chart, Revenue by Category pie chart, Category slicer

4.2 Data Model in Power BI

Power BI auto-detected all relationships upon loading the six tables from PostgreSQL. The model follows a star schema with `Order_Items` as the central fact table.

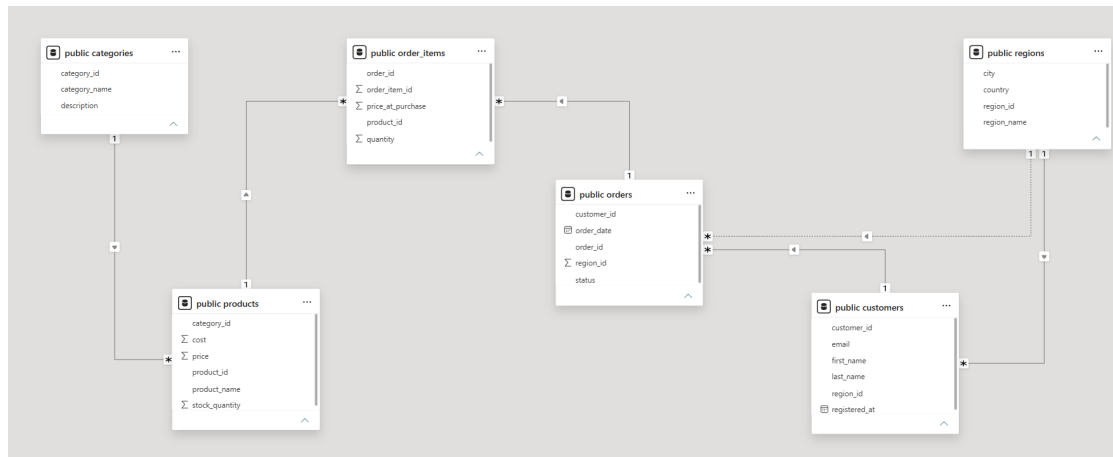


Figure 4.1. Power BI Model View showing all six tables and their auto-detected relationships.

4.3 Design Theme

The dashboard uses a custom colour palette inspired by the RetailPulse brand:

Table 4.2. RetailPulse Power BI colour palette

Element	Hex Code	Usage
Page background	#1B2A4A	Dark navy canvas background
Card background	#243860	KPI card and visual backgrounds
Accent / Charts	#00C4FF	Bars, lines, accent bars, borders
Header banner	#00C4FF	Top banner across all pages
Text	#FFFFFF	All titles and labels

4.4 Dashboard Pages

4.4.1 Page 1 — Executive Summary

The Executive Summary page provides a high-level overview of overall sales performance. It features four KPI cards showing total revenue, total orders, total customers, and average order value. A line chart displays the monthly sales trend over the full data period. A date range slicer allows filtering all visuals by time period.

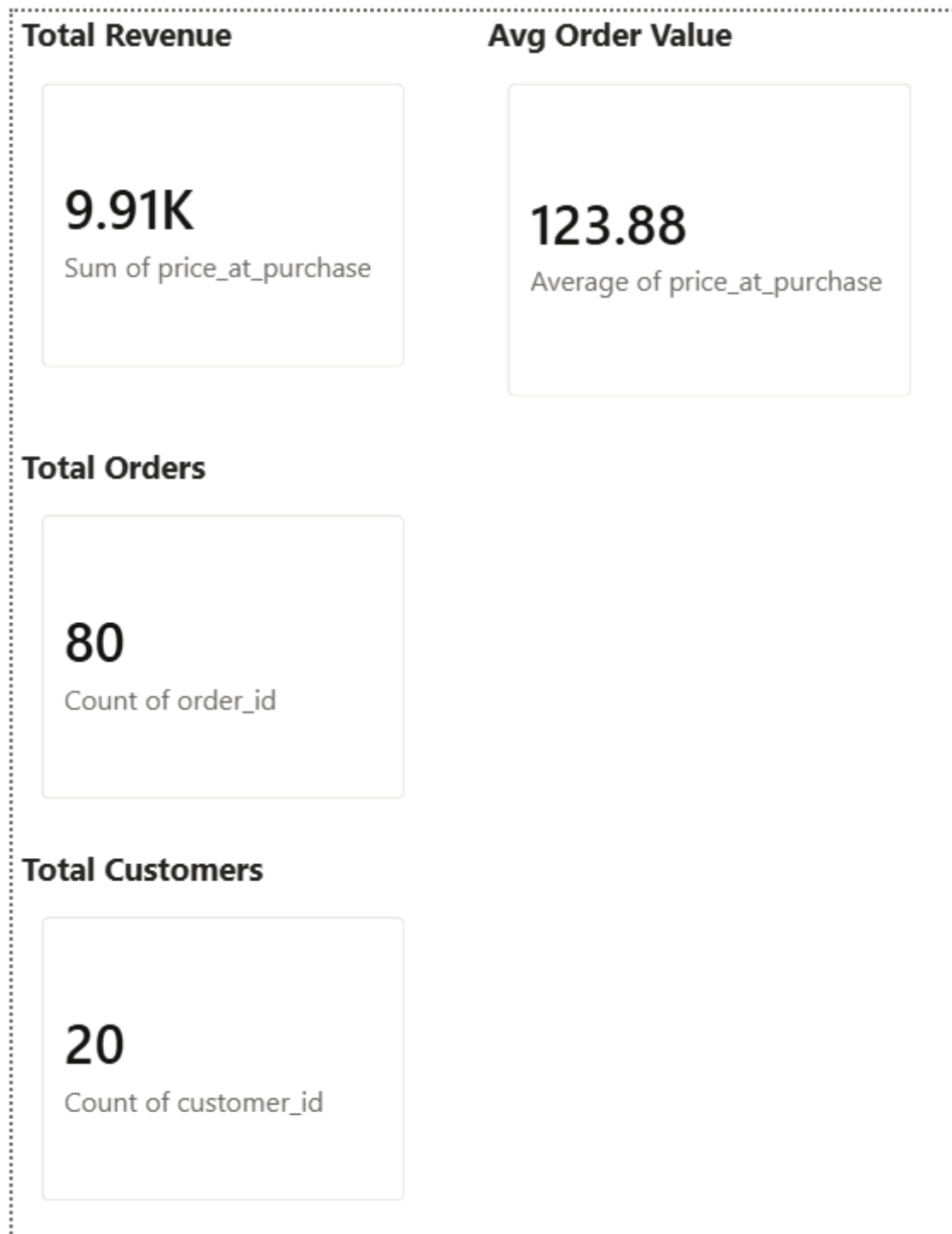


Figure 4.2. Initial layout of the Executive Summary page showing KPI cards and Monthly Sales Trend before formatting.

After applying the RetailPulse colour scheme and formatting, the page looks like this:

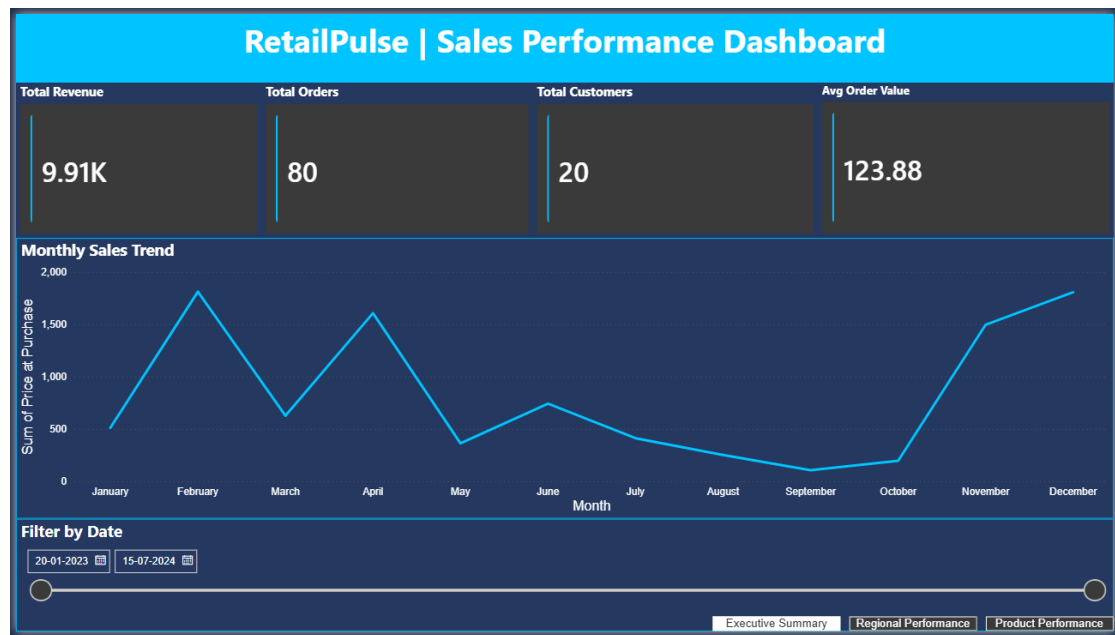


Figure 4.3. Final Executive Summary page — KPI cards showing total revenue of 9.91K, 80 orders, 20 customers, and average order value of 123.88. Monthly Sales Trend shows peaks in February, April, November, and December.

4.4.2 Page 2 — Regional Performance

The Regional Performance page provides a geographical breakdown of sales. A horizontal bar chart ranks all eight regions by revenue, with North America West leading at 3.3K. A column chart shows revenue by country, with the USA as the top contributor at 4.4K. A dropdown slicer allows filtering by region.

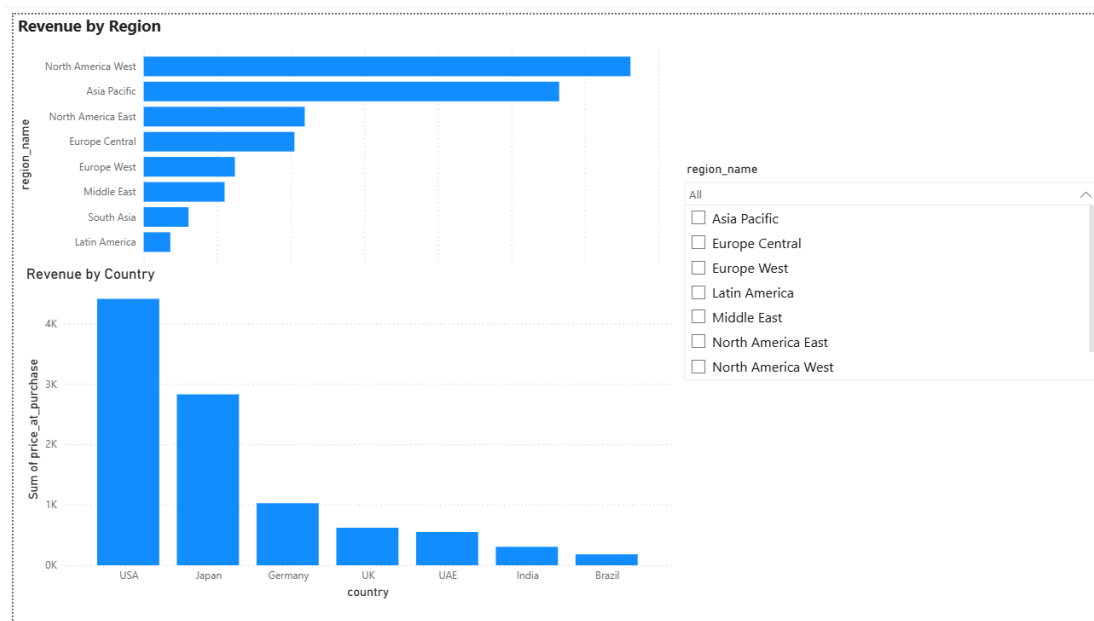


Figure 4.4. Initial layout of the Regional Performance page before formatting.

After applying the RetailPulse colour scheme and formatting, the page looks like this:

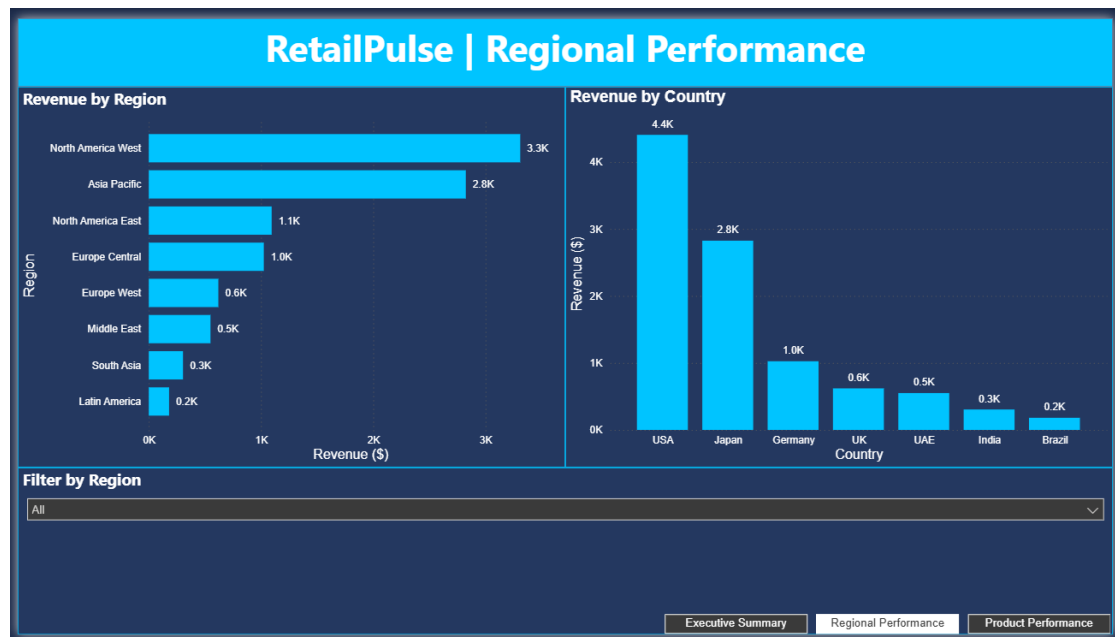


Figure 4.5. Final Regional Performance page — North America West leads with 3.3K revenue, followed by Asia Pacific at 2.8K. USA dominates country-level revenue at 4.4K.

4.4.3 Page 3 — Product Performance

The Product Performance page provides a product and category-level analysis. A horizontal bar chart ranks the top products by revenue, with Laptop Pro 15 leading at 2.6K. A pie chart shows revenue distribution by category, with Electronics dominating at 55.5% of total revenue. A dropdown slicer allows filtering by product category.

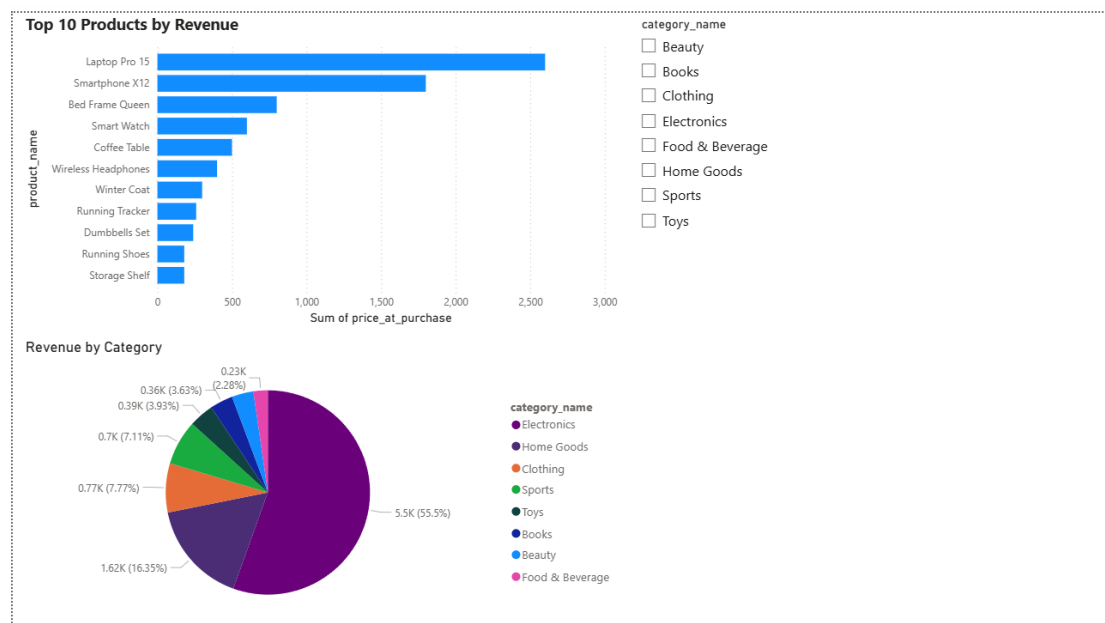


Figure 4.6. Initial layout of the Product Performance page before formatting.

After applying the RetailPulse colour scheme and formatting, the page looks like this:

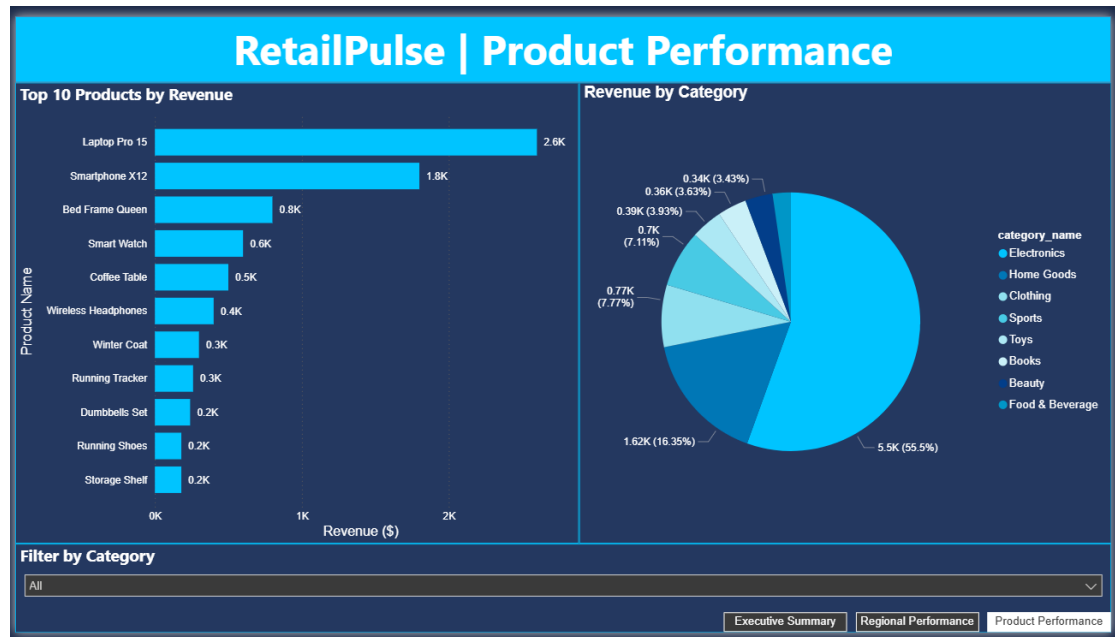


Figure 4.7. Final Product Performance page — Laptop Pro 15 is the top revenue product at 2.6K. Electronics accounts for 55.5% of total category revenue.

4.5 Key Insights from the Dashboard

Top Insights from RetailPulse Dashboard:

1. **Total Revenue:** \$9,910 generated across 80 orders from 20 unique customers.
2. **Top Region:** North America West leads with \$3,300 in revenue, followed by Asia Pacific at \$2,800.
3. **Top Country:** USA generates the highest revenue at \$4,400, nearly double Japan at \$2,800.
4. **Top Product:** Laptop Pro 15 is the highest revenue product at \$2,600, followed by Smartphone X12 at \$1,800.
5. **Top Category:** Electronics dominates with 55.5% of total revenue, highlighting a strong technology product demand.
6. **Sales Trend:** Sales peak in February, April, November, and December — suggesting seasonal demand patterns around holidays and new year periods.
7. **Average Order Value:** \$123.88 per order indicates a moderate-to-high value transaction base.

Chapter 5

Conclusion

5.1 Summary

The RetailPulse Sales Performance Dashboard successfully demonstrates an end-to-end data analytics solution built using industry-standard tools and best practices. The project spans the complete data lifecycle — from relational database design and data population, through backend API development, to interactive visualisation in Power BI.

The key achievements of this project are:

- A normalised PostgreSQL database with six interrelated tables, proper constraints, and foreign key relationships storing realistic sales data across 8 regions, 8 categories, 40 products, 20 customers, 40 orders, and 80 order line items.
- A RESTful backend API built with Node.js and Express.js exposing 8 analytical endpoints, providing aggregated sales metrics to any connected client.
- A professional, interactive three-page Power BI dashboard with a consistent dark navy and cyan design theme, featuring KPI cards, trend charts, regional and product breakdowns, and interactive slicers and page navigation.

5.2 Challenges and Learnings

Table 5.1. Key challenges encountered and how they were resolved

Challenge	Resolution
Python 3.14 venv incompatibility on Windows	Switched API framework to Node.js/Express.js which had no such issues
PostgreSQL password with special characters	Wrapped password in double quotes in the <code>.env</code> file
PowerShell <code>curl</code> syntax differences	Used <code>Invoke-WebRequest</code> as the PowerShell equivalent
Power BI auto-detecting incorrect relationships	Manually verified and corrected cardinality via Edit Relationship dialog

5.3 Future Enhancements

Given more time, the following enhancements would further improve the RetailPulse solution:

- **Larger dataset** — generating thousands of orders using Python scripts for more statistically meaningful trend analysis.
- **API authentication** — adding JWT-based authentication to secure the API endpoints.
- **Real-time data** — implementing a data refresh schedule in Power BI Service for live dashboard updates.
- **Forecasting** — adding Power BI's built-in forecasting feature to the Monthly Sales Trend chart to project future revenue.
- **Profit margin analysis** — using the `cost` column in the Products table to add gross margin visualisations.
- **Publishing** — publishing the dashboard to Power BI Service for web-based access and sharing.

5.4 Final Remarks

This capstone project provided hands-on experience across the full data engineering and analytics pipeline. Building RetailPulse from a blank PostgreSQL database to a polished, interactive Power BI dashboard reinforced the interconnectedness of database management, API development, and data visualisation — core competencies for any data professional.

The complete codebase, SQL scripts, and Power BI file for this project are available in the project repository.